ACEest Fitness & Gym DevOps Implementation Report

Student Name: PRATHEUSHA KK

BITS ID: 2024TM93723

Work Integrated Learning Programmes Division (WILPD),

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI,

VIDYA VIHAR, PILANI, RAJASTHAN - 333031.

(April 2026)

1. INTRODUCTION

This report presents a complete DevOps implementation using the ACEest Fitness & Gym application. The objective is to demonstrate how modern DevOps practices integrate development, testing, containerization, CI/CD pipelines, and cloud deployment into a unified workflow.

2. PROJECT OVERVIEW

The ACEest Fitness & Gym application is a Flask-based web application that provides both REST APIs and a graphical user interface. It allows users to browse fitness programs and estimate calorie requirements based on user inputs.

3. TECHNOLOGY STACK

Tool	Purpose
Flask	Backend development
Docker	Containerization
Jenkins	CI/CD pipeline
SonarQube / Sonar CLI	Static analysis
JFrog	Artifact storage
GitHub	Version control
Azure	Cloud deployment
GitHub Actions	Repository-native CI

4. GITHUB FLOW STRATEGY

A GitHub Flow strategy was implemented where the main branch is always production-ready. Feature branches are created for development, and Pull Requests ensure code quality through reviews and automated checks.

GitHub Repository URL: https://github.com/Pratheusha-kk/aceest-app/tree/main

4.1 STABLE MAIN AS THE SINGLE SOURCE OF TRUTH

- The `main` branch represents the deployable, production-ready state of the ACEest Fitness & Gym application.
- Only code that has passed reviews, automated tests, and quality checks is allowed to reach `main`.
- Jenkins is configured so that production-grade actions for pushing Docker images to the prod Artifactory repo only run when `BRANCH_NAME == "main"` after a successful merge into `main`.

4.2 SHORT-LIVED FEATURE BRANCHES FOR ALL WORK

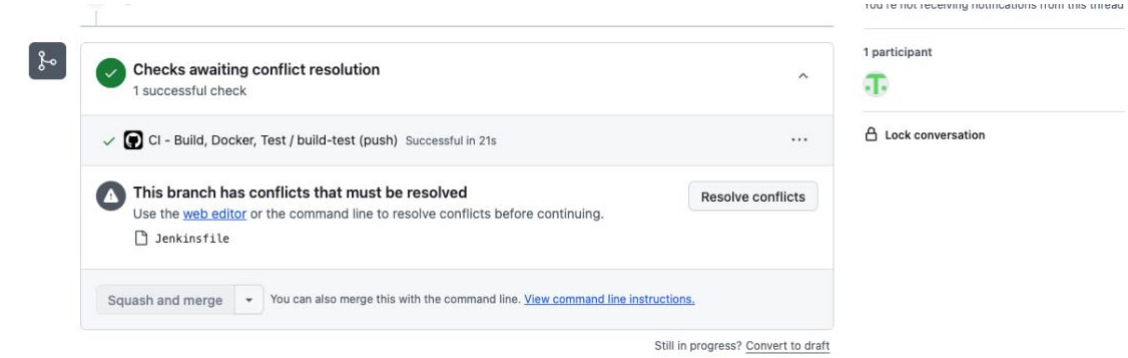
- Any new work—whether it's a feature in `app.py`, a UI change in `templates/`, or configuration updates in `Dockerfile` / `Jenkinsfile`— is done on a dedicated feature branch created from main, for example:
  - `nutrition_feature`
  - `pipeline_branch`
  - `sonar_corretion`
- No direct merges to '`main` are allowed. Instead, the following steps are used:

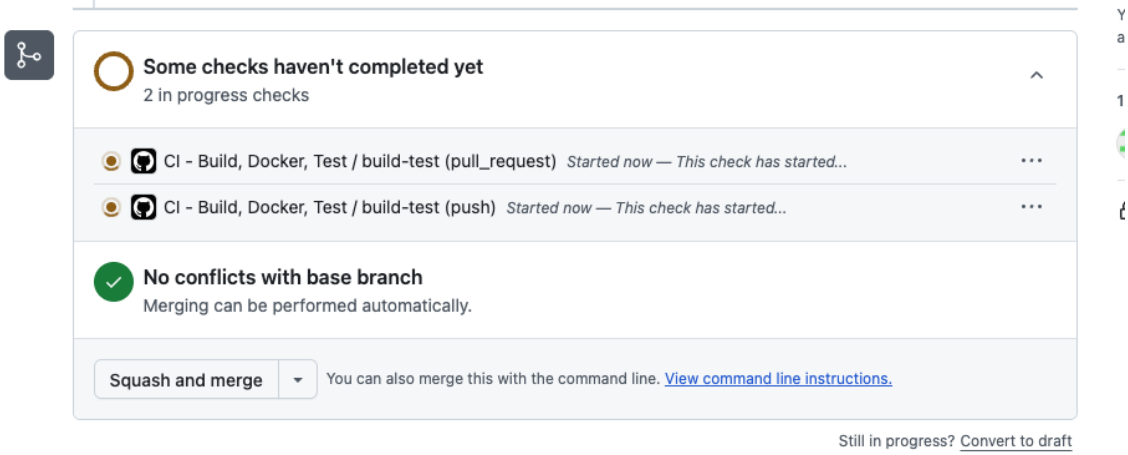
```
git checkout main && git pull

git checkout -b some-change

# implement changes

git push -u origin some-change
```

This keeps `main` clean and ensures all changes are isolated until they are reviewed.

4.3 PULL REQUESTS AS THE INTEGRATION GATE

- When a feature branch is ready, a Pull Request (PR) is opened which facilitates:
  - Code review comments and discussions.
  - Status checks from Jenkins and GitHub Actions.
  - Visibility into what exactly will be merged into `main`.
  - As part of the GitHub Flow, the feature branch is updated as needed (e.g., rebased or merged with the latest `main`) until the PR is green and approved.
  - Every push to a PR branch triggers:
  - The GitHub Actions workflow (build, Docker image assembly, Pytest).
  - The Jenkins pipeline for deeper checks and, for `main`, deployment.

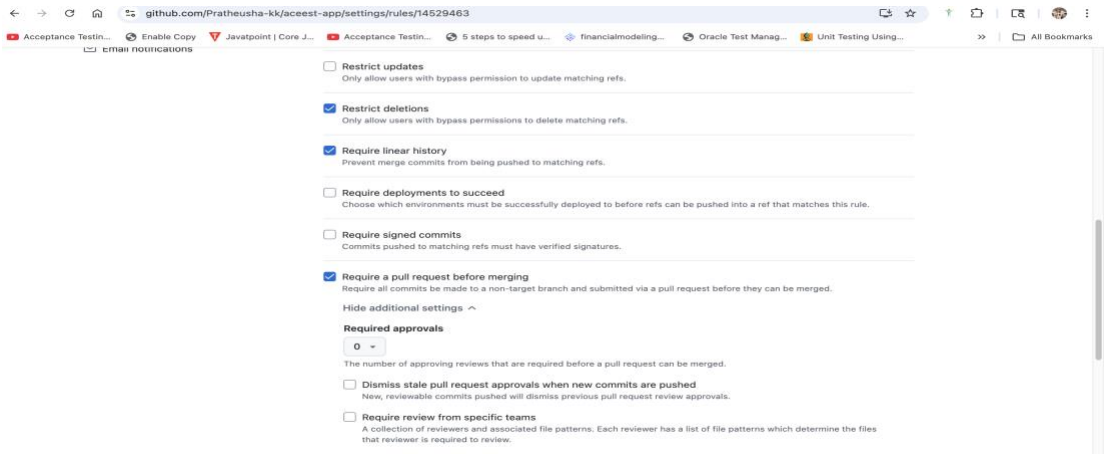




4.4 BRANCH PROTECTION RULES ON `MAIN`

To enforce this workflow and avoid accidental or unreviewed changes to `main`, Branch Protection Rules are configured in GitHub:

- Require pull request reviews before merging.
- Require status checks to pass before merging:
    - A PR cannot be merged unless the CI pipelines succeed, meaning:
    - Pytest tests have passed.
    - (Optionally) UI tests have passed.
    - Static analysis, Docker build, and smoke tests are green.



4.5 INTEGRATION WITH THE JENKINS PIPELINE

The GitHub Flow strategy aligns directly with the branch-based behaviour in the `Jenkinsfile`:

- For feature branches (`BRANCH_NAME != "main"`):
  - Jenkins runs the full pipeline (Sonar, tests, Docker build, smoke tests, stage push, stage deployment).
  - This gives early feedback and lets developers validate changes in a staging-like environment.
  - Production push is intentionally skipped because the MR is not yet merged.
  - When the PR is merged into `main`:
- GitHub creates a new commit on `main`.
- Jenkins picks up this `main` commit and runs the pipeline with `BRANCH_NAME == "main"`.
- The prod push stage (Docker: Push Image to Artifactory (Prod)) executes, producing a versioned `aceest-app` image in the prod Artifactory repo.

Thus, GitHub Flow + Branch Protection Rules + CI pipelines guarantee that only reviewed, tested, and pipeline-validated changes trigger the production image push and subsequent production deployments.

5. APPLICATION DEVELOPMENT

The application exposes endpoints such as `/`, `/health`, `/programs`, and `/estimate-calories`. It also includes GUI routes for user interaction, ensuring both API and UI functionality.

5.1 REPOSITORY STRUCTURE (KEY FILES AND FOLDERS)

File/Folder	Details
`app.py`	Main Flask application defining JSON API endpoints and the HTML GUI routes.
`Dockerfile`	Instructions to build the container image for the ACEest Fitness & Gym app.
`Jenkinsfile`	Declarative Jenkins pipeline (SonarQube analysis, unit/UI tests, Docker build, push to JFrog, Azure deploy).
`requirements.txt`	Python dependencies for running the app.
`sonar-project.properties`	SonarQube project configuration (project key, name, sources, tests).
`templates/`	Jinja2 HTML templates for the web GUI (`index.html`, `programs.html`, `program_detail.html`, `calories.html`, `layout.html`).
`static/`	Static assets such as `styles.css` used by the GUI.
`tests/`	Unit and integration tests for the Flask API (e.g. `test_app.py`, `test_integration.py`).
`ui-tests/`	Behave + Selenium UI test suite and its own `requirements.txt`.

.github/workflows/main.yml	GitHub Actions CI pipeline definition (build, Docker image assembly, Pytest tests).
README.md	Documentation for setup, endpoints, testing, Docker usage, and CI/CD pipeline.

5.2 FLASK APPLICATION

The ACEest Fitness & Gym app is built with Flask and exposes both JSON APIs and an HTML GUI.

Key routes include:

GET/	API health message: confirms the backend is running.
GET/health	Lightweight health probe used by smoke tests and UI-tests to verify the container is healthy.
GET/programs and GET/programs/<program_name>	JSON endpoints that return the catalog of fitness programs and detailed program info (description, workout, diet, calorie factor).
GET/estimate-calories	JSON API that estimates daily calories based on a chosen program and weight (kg).
GET/gui	Web GUI home page with navigation tiles to browse programs and use the calorie estimator.
GET/gui/programs and GET/gui/programs/<program_name>	HTML pages listing all programs and showing detailed workout + diet for a single program.
GET/gui/calories	Web form for selecting a program and entering weight to calculate and display calorie estimates in the browser.

These routes share the same underlying logic and data model, providing both machine-readable APIs and a human-friendly UI.

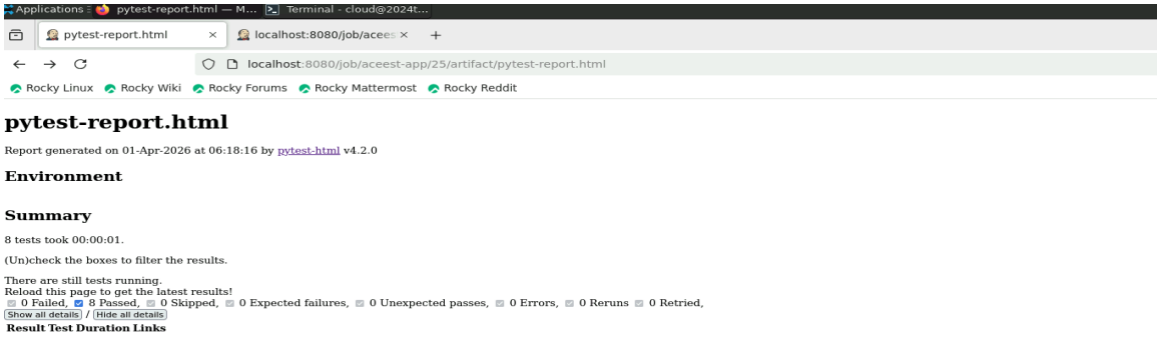
6. TESTING STRATEGY

Testing is performed using Pytest for unit and integration testing. Selenium is used for UI testing to validate end-to-end functionality.

6.1 PYTEST FOR UNIT AND INTEGRATION TESTING

Local code validation is done using Pytest, which is also executed in the Jenkins pipeline before building and deploying the application. The tests/ directory contains unit and integration tests that:

- Import the Flask app from app.py and use Flasks test client to call endpoints like /, /health, /programs, /programs/<program_name>, and /estimate-calories.
- Verify correct HTTP status codes (e.g., 200 for valid requests, 400/404 for invalid program names or weights).
- Assert JSON response structure and key fields (e.g., message, status, calories_kcal, calorie_factor).



6.2 UI TESTING WITH SELENIUM

- UI tests in `ui-tests/` use Behave + Selenium to automate browser-based verification of key pages and navigation tabs.
- These tests validate that the deployed GUI behaves correctly from the users perspective.

7. DOCKER CONTAINERIZATION

The application is containerized using a multi-stage Docker build. This reduces image size and improves security by running the application as a non-root user.

7.1 DOCKERFILE IMPLEMENTATION

The application is containerized using a secure, multi-stage `Dockerfile` that optimizes image size and follows best practices:

Builder stage:

- Uses a lightweight Python base image to install dependencies from `requirements.txt`.
- Leverages layer caching by copying `requirements.txt` and installing packages before copying the rest of the source code, so that dependency layers are reused when only app code changes.

Runtime stage:

- Starts from a clean Python image and copies only the built application and installed dependencies from the builder stage, keeping the final image smaller and free of build tools.
- Runs the app as a non-root user (created in the image) to follow the principle of least privilege and reduce the impact of potential container escapes or vulnerabilities.
- Exposes port 5000 and sets the default command to start the Flask app (`python app.py`).

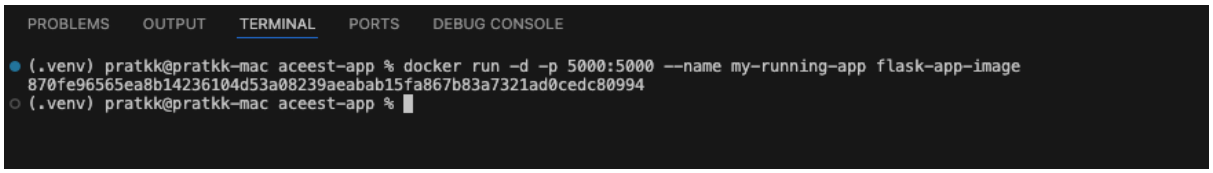
This approach yields a secure, minimal Docker image that is efficient to build and safe to run in CI/CD and production.

7.2 LOCAL IMAGE VERIFICATION

Before integrating into the pipeline, the Docker image was built and verified locally. With the container running, the application was validated via:

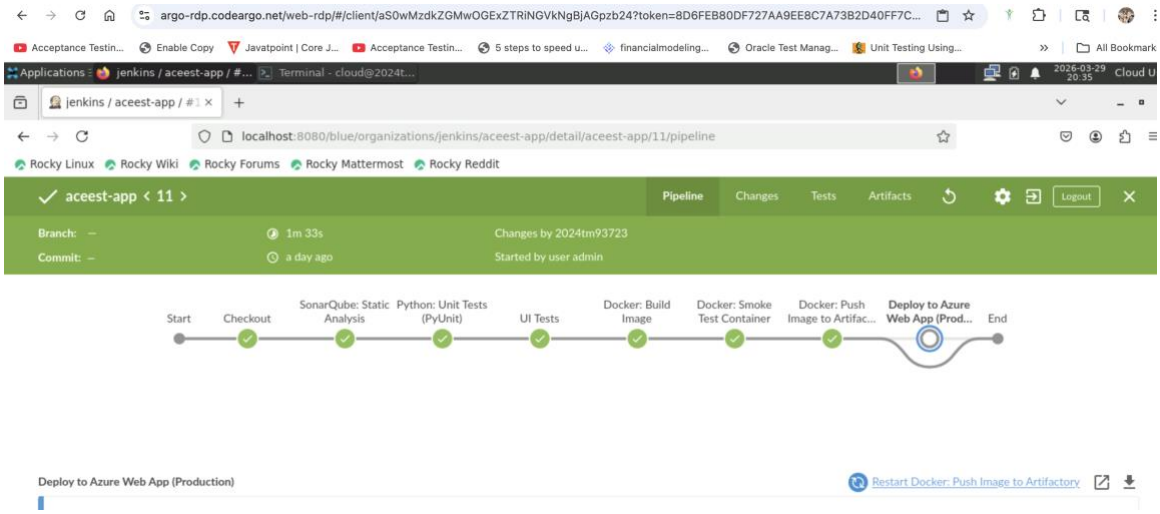
http://localhost:5000/	API health message.
http://localhost:5000/health	health probe.
http://localhost:5000/programs	JSON list of programs.
http://localhost:5000/gui	HTML GUI home page.

Successful responses from these endpoints confirmed that the containerized application works as expected and that the Dockerfile is correct.



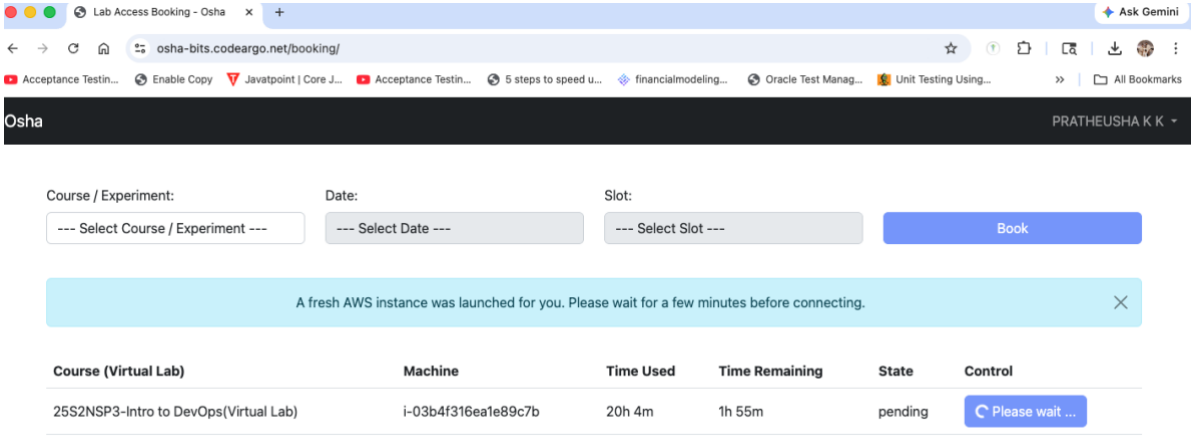
8. JENKINS CI PIPELINE

Jenkins automates the CI/CD process including code checkout, testing, static analysis, Docker image creation, and deployment. Poll SCM is used instead of webhooks due to network constraints.



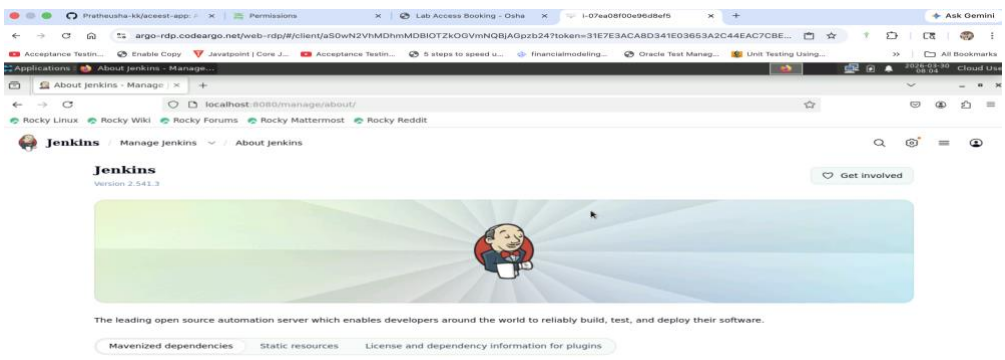
8.1 JENKINS SETUP

Jenkins was already installed on the Linux VM assigned for the DevOps virtual lab and configured as the primary build server for the assignment.



Key steps performed (high-level):

1. Updated Java and Jenkins (2.541.3).



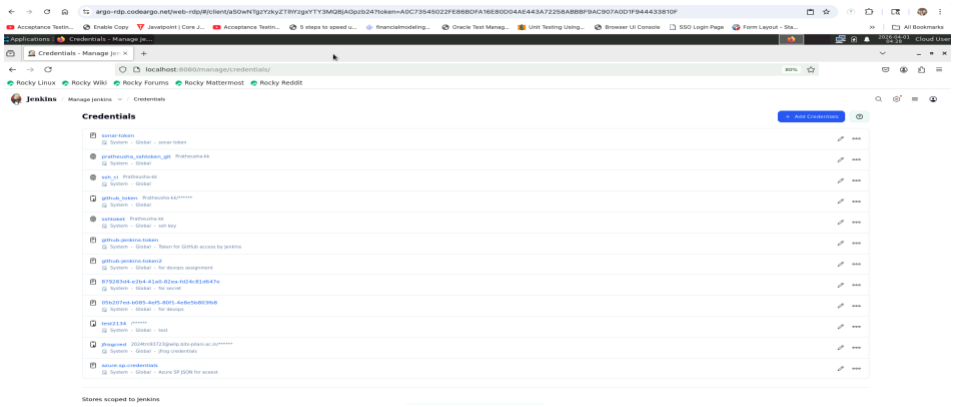
2. Started the Jenkins service and accessed it via http://localhost:8080.

   Initially, an attempt was made to expose the Jenkins endpoint externally so that it could be accessed outside the virtual machine environment. However, this approach was not successful because the required port was not accessible on the Azure virtual lab instance due to network restrictions. To overcome this, tools like Cloudflare Tunnel were explored to create a public URL. While this worked temporarily, the VM instance did not consistently retain the same endpoint (likely due to restarts or session resets), resulting in frequent URL changes. This made the setup unreliable for continuous integration workflows.

   To ensure stability and consistency, Jenkins and the application were instead hosted on `localhost`, and external exposure was avoided. Since webhook-based triggers require a publicly accessible endpoint, they could not be used in this setup. As an alternative, the ****Poll SCM**** mechanism in Jenkins was configured. This allows Jenkins to periodically check the GitHub repository for changes instead of relying on webhooks.

   Jenkins is configured to monitor the `aceest-app` repository and automatically trigger the pipeline when changes are detected.

3. Installed required plugins like Git, Pipeline, Sonar, Docker.
4. Configured credentials for GitHub access, JFrog, and Azure Service Principal.

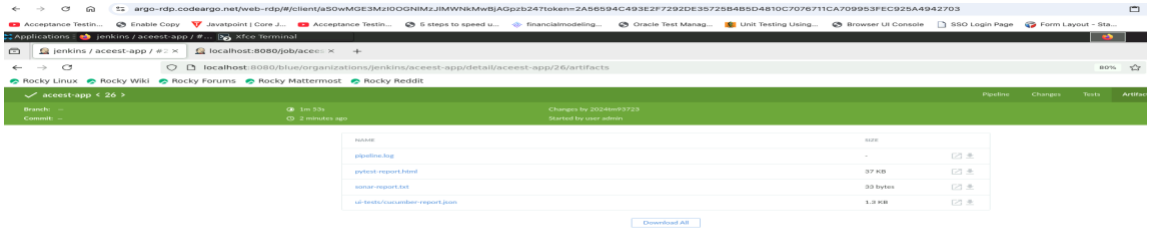


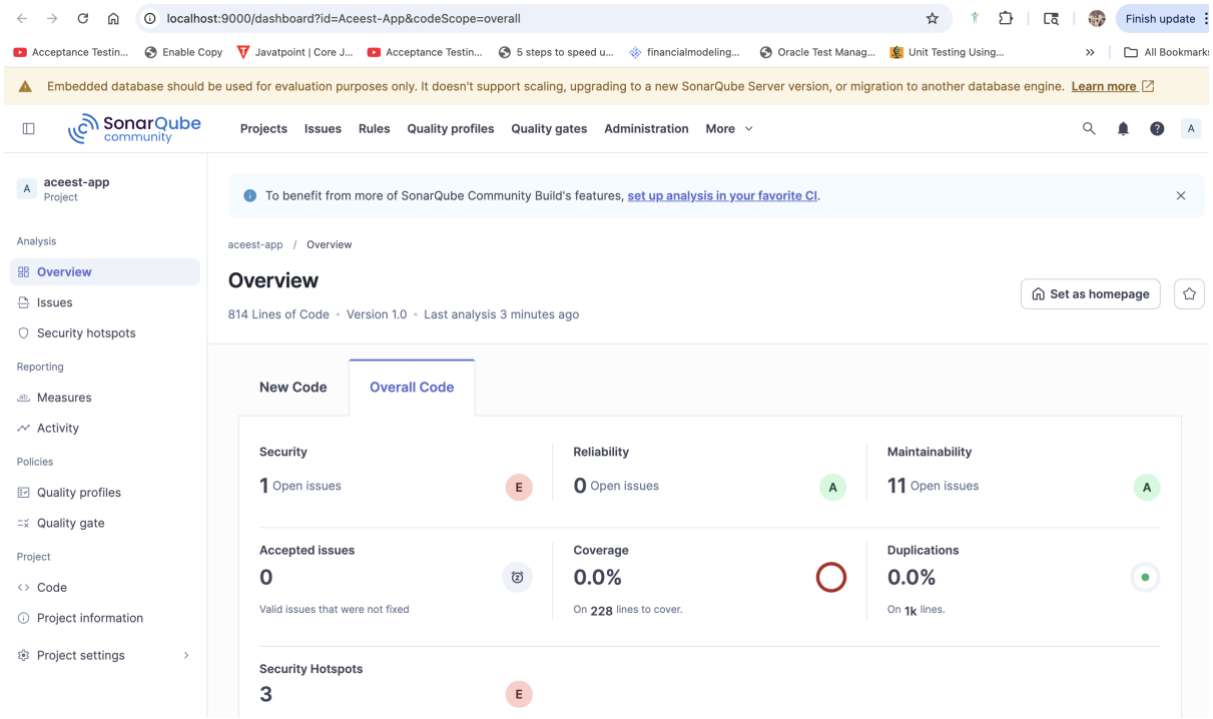
8.2 STATIC CODE ANALYSIS

For static code analysis, instead of using a full-fledged SonarQube server with a web-based dashboard, a lightweight CLI-based approach was adopted due to the same network and environment constraints. Running a SonarQube server would require exposing a UI endpoint (typically on port 9000), which was not feasible in the given setup. The Azure virtual machine environment restricted external access to these ports, making it impossible to view or interact with a central SonarQube UI.

```
[cloud@2024tm93723 ~]$ sonar-scanner --version
INFO: Scanner configuration file: /opt/sonar-scanner/conf/sonar-scanner.properti
es
INFO: Project root configuration file: NONE
INFO: SonarScanner 5.0.1.3006
INFO: Java 21.0.10 Red Hat, Inc. (64-bit)
INFO: Linux 5.14.0-503.21.1.el9_5.x86_64 amd64
```

Therefore, the Sonar CLI (or sonar-secrets plugin) was used to perform local static analysis directly within the Jenkins pipeline. The analysis is triggered using the `sonar-scanner` command with configurations defined in `sonar-project.properties`. Instead of sending results to a remote server, the scan outputs are exported as a text report and archived.





9. GITHUB ACTIONS CI PIPELINE

To align with modern DevOps practices and the assignment requirement for an automated CI pipeline using GitHub Actions, a workflow is defined in the repository at:

`.github/workflows/main.yml`

This pipeline provides fast, repository-native CI on every code change and complements the Jenkins pipeline.

9.1 TRIGGERS

The workflow is configured to run on:

- `push` events (to relevant branches such as `main` and feature branches).
- `pull_request` events targeting `main`.

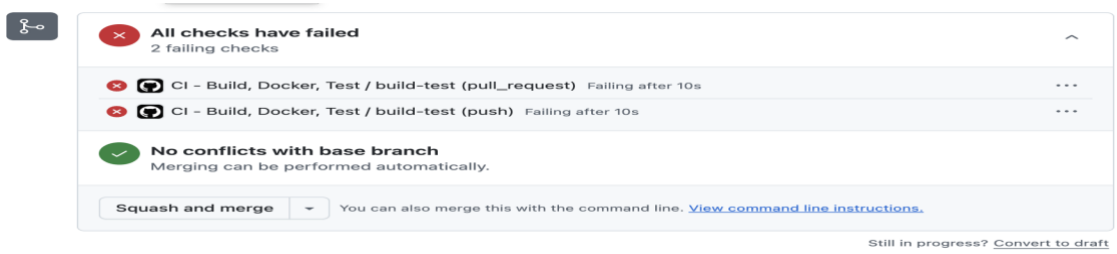
This ensures that every commit and PR automatically triggers the GitHub Actions pipeline.

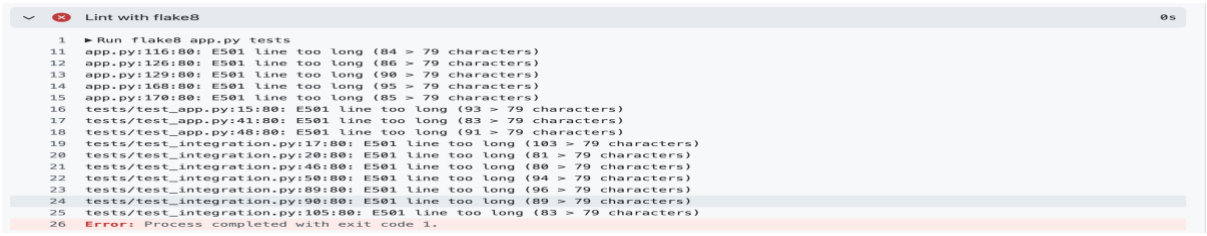
9.2 JOBS AND STAGES

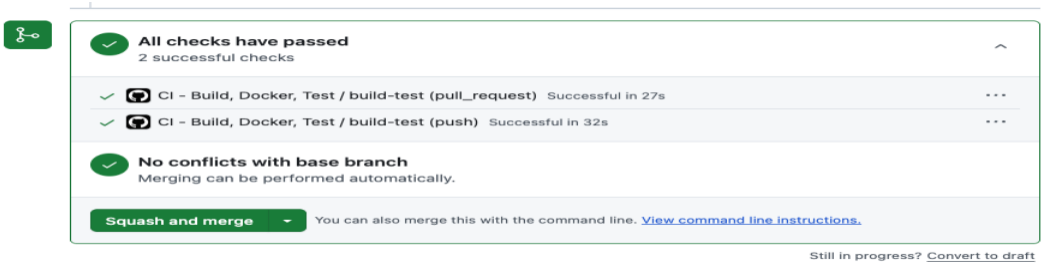
The GitHub Actions workflow includes the following logical stages:

1. **Build & Lint**

- Check out the repository.
- Set up Python (e.g., using `actions/setup-python`).
- Install dependencies from `requirements.txt`.
- Optionally run a linter (e.g., `flake8`) to catch syntax and style issues early.
- This stage verifies that the Python codebase is structurally sound before proceeding.





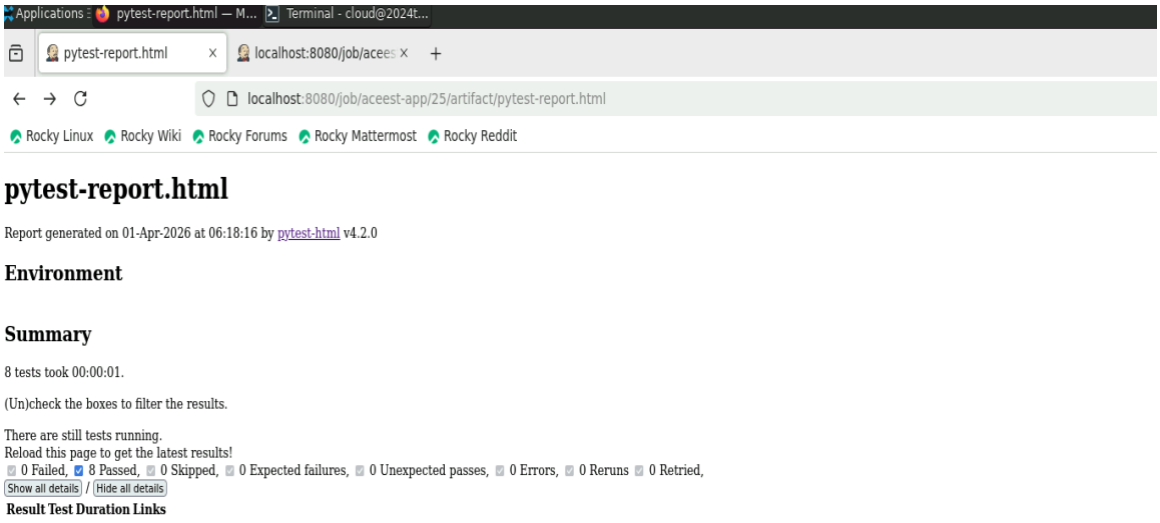


2. **Docker Image Assembly**

- Build the Docker image using the project `Dockerfile`.
- This validates that the `Dockerfile` is correct and that the application can be successfully containerized directly from GitHub.
- The same `Dockerfile` is used later in Jenkins, ensuring consistency across CI tools.

3. **Automated Testing (Pytest)**

- Run the `Pytest` suite either:
    - Directly on the host runner, or
    - Inside the built Docker container (for environment parity).
    - Tests exercise the Flask endpoints and business logic described in the Testing Strategy section.



9.3 RELATIONSHIP WITH JENKINS

The GitHub Actions pipeline and Jenkins pipeline play complementary roles:

GitHub Actions

- Provides fast feedback on every push/PR within GitHub itself.
- Focuses on build, linting, Docker build validation, and Pytest execution.
- Acts as a pre-merge quality gate visible directly in Pull Requests.

Jenkins

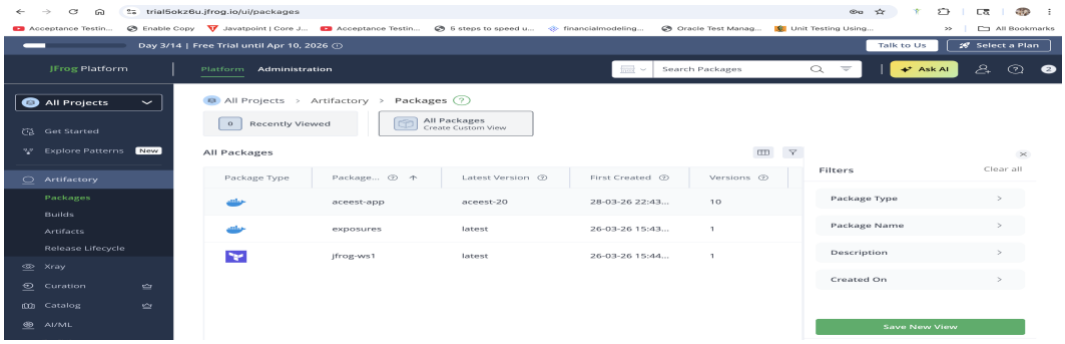
- Performs deeper pipeline tasks once changes are integrated:
    - Static analysis using Sonar CLI.
    - Docker image tagging and pushing to JFrog Artifactory (stage/prod repos).
    - Deployment to Azure Web App using the container image.
    - Acts as the main orchestrator for artifact promotion and deployment.

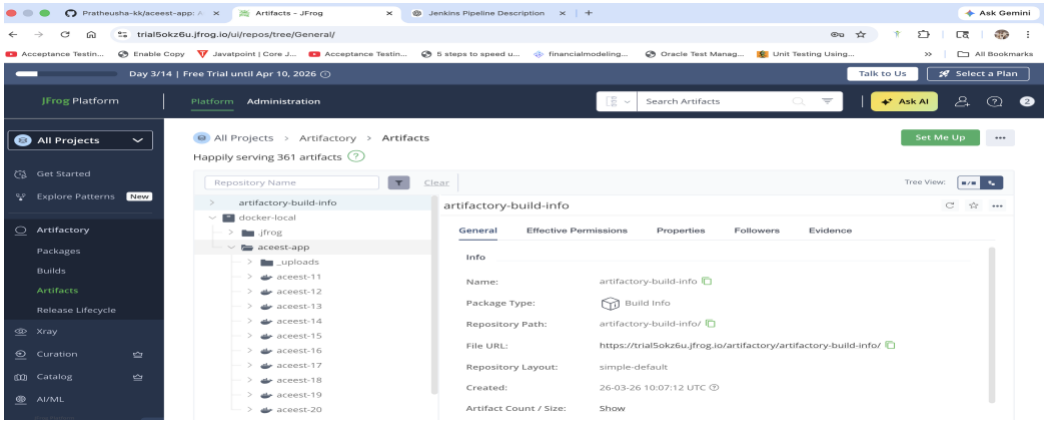
In Pull Requests, both GitHub Actions checks and Jenkins build status must be green before merging into `main`, ensuring a robust CI/CD workflow.

10. UPLOAD IMAGE TO ARTIFACTORY

A cloud-based artifact repository was required to store and manage Docker images generated during the CI/CD process. For this purpose, the trial version of JFrog Artifactory was used:

Artifactory URL (14 days Free tier) - `https://trial5okz6u.jfrog.io/`





Two separate repositories were configured:

- `docker-local` → for staging (non-`main` branches).
- `docker-prod` → for production (`main` branch).

This separation ensures proper environment isolation and supports a structured promotion strategy from staging to production.

11. DEPLOYMENT TO AZURE

On the deployment side, the Jenkins agent is prepared with the Azure CLI and a dedicated Azure Service Principal (SP) for secure, non-interactive authentication.

11.1 AZURE AUTHENTICATION

- Azure CLI is installed on the Jenkins node so that commands like `az login`, `az account set`, and `az webapp config container set` can be executed from the pipeline.
- A Service Principal is created in Azure AD (with least-privilege access to the target subscription and resource group). Its details (`tenantId`, `clientId`, `clientSecret`, `subscriptionId`) are stored in Jenkins as a Secret Text credential (e.g. `azure-sp-credentials`), and injected into the pipeline as JSON.
- In the `Jenkinsfile`, this JSON is written to a temp file and parsed with `jq` to extract the tenant, app ID, password, and subscription ID. These are then used to run:

```
az login --service-principal \
  --username "${APPID}" \
  --password "${PASSWORD}" \
  --tenant "${TENANT}"
az account set --subscription "${SUBSCRIPTION}"
```

This ensures all Azure operations (Web App configuration, container deployment, restarts) are authenticated and auditable using the SP identity rather than a personal account.

11.2 AZURE DEPLOY STAGE IN JENKINS

The `Jenkinsfile`s "`Deploy to Azure Web App (Stage)`" stage runs when `BRANCH_NAME != "main"`.

It pulls in two sets of credentials:

- The Azure Service Principal JSON (`AZURE_SP_JSON`).
- The JFrog Docker registry credentials (`REG_USER, REG_PASS`) used by Azure Web App to pull the container.

It logs into Azure with the SP, sets the correct subscription, and constructs the full image reference:

bash

```
IMAGE="${AZURE_CONTAINER_REGISTRY_SERVER}/${AZURE_CONTAINER_IMAGE_NAME}:${deployTag}"
# e.g. trial5okz6u.jfrog.io/docker-local/aceest-app:aceest-123
```

It then configures the Azure Web App for Containers to use this image and private registry:

```
az webapp config container set \
  --resource-group "${AZURE_RESOURCE_GROUP}" \
```

```
  --name "${AZURE_WEBAPP_NAME}" \
  --docker-custom-image-name "${IMAGE}" \
  --docker-registry-server-url "https://${AZURE_CONTAINER_REGISTRY_SERVER}" \
  --docker-registry-server-user "${REG_USER}" \
  --docker-registry-server-password "${REG_PASS}"
```

Finally, it sets application settings (port and startup timeout) and restarts the Web App:

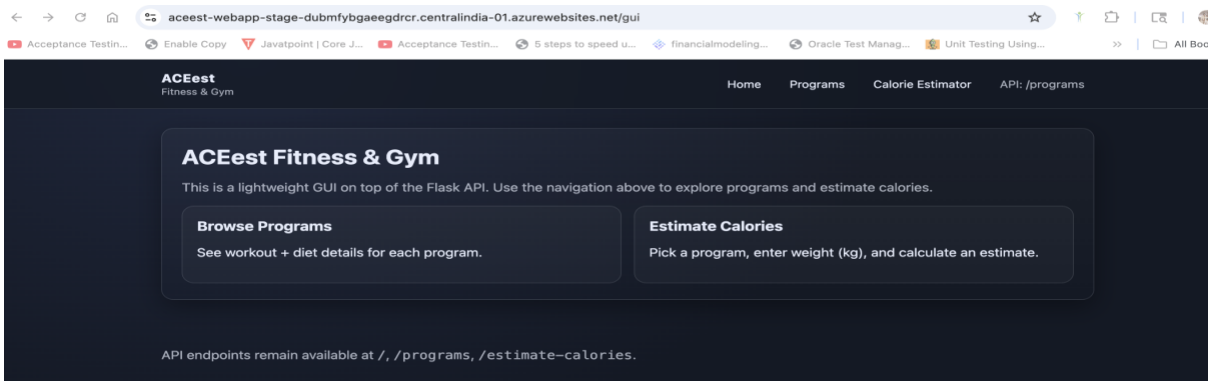
```
az webapp config appsettings set \
  --resource-group "${AZURE_RESOURCE_GROUP}" \
  --name "${AZURE_WEBAPP_NAME}" \
  --settings WEBSITES_PORT=5000 WEBSITES_CONTAINER_START_TIME_LIMIT=1000


az webapp restart --resource-group "${AZURE_RESOURCE_GROUP}" --name
"${AZURE_WEBAPP_NAME}"
```

This sequence makes Azure pull the latest container image from JFrog and run it as the backend for the Web App service.

11.3 LIVE APPLICATION ENDPOINTS

Once deployed, the ACEest Fitness & Gym application is available through Azure Web App URLs. Using the actual Web App name and region, the base URL looks like:

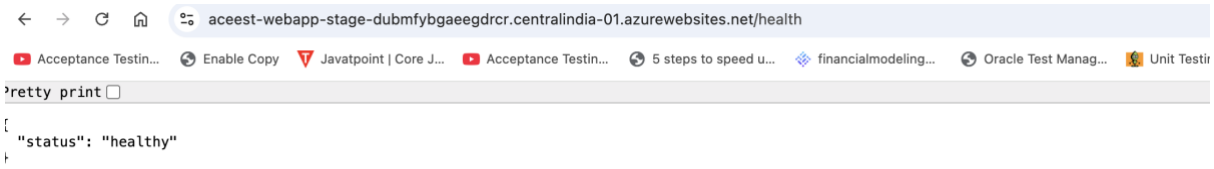
Stage Web App: https://aceest-webapp-stage-dubmfybgaeegdrcr.centralindia-01.azurewebsites.net/gui



Key live endpoints (examples):

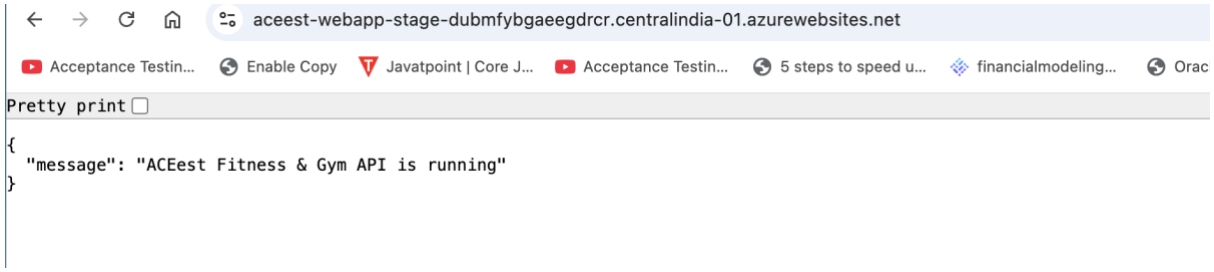
Health check (API): https://aceest-webapp-stage-dubmfybgaeegdrcr.centralindia-01.azurewebsites.net/health

Returns JSON like `{ "status": "healthy" }` if the container is running.

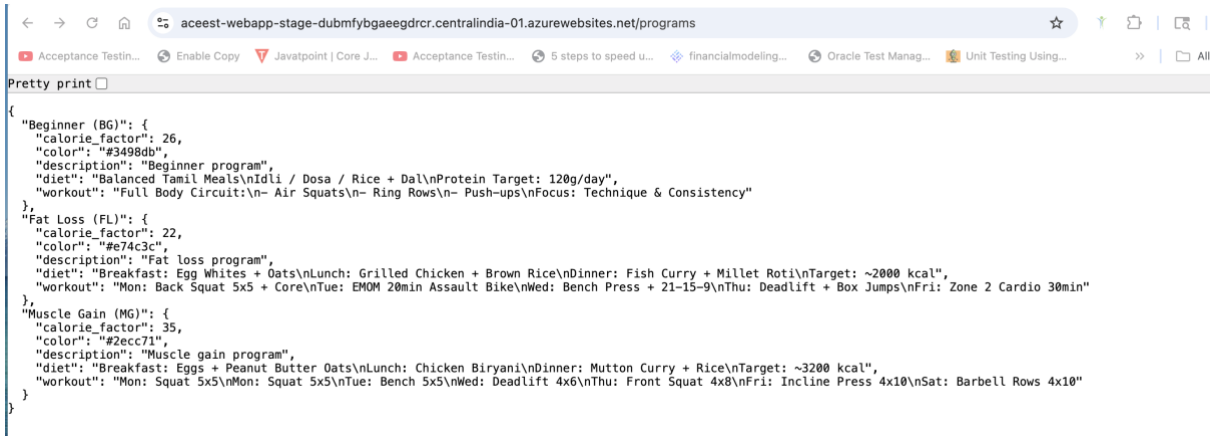


Root API endpoint: https://aceest-webapp-stage-dubmfybgaeegdrcr.centralindia-01.azurewebsites.net/

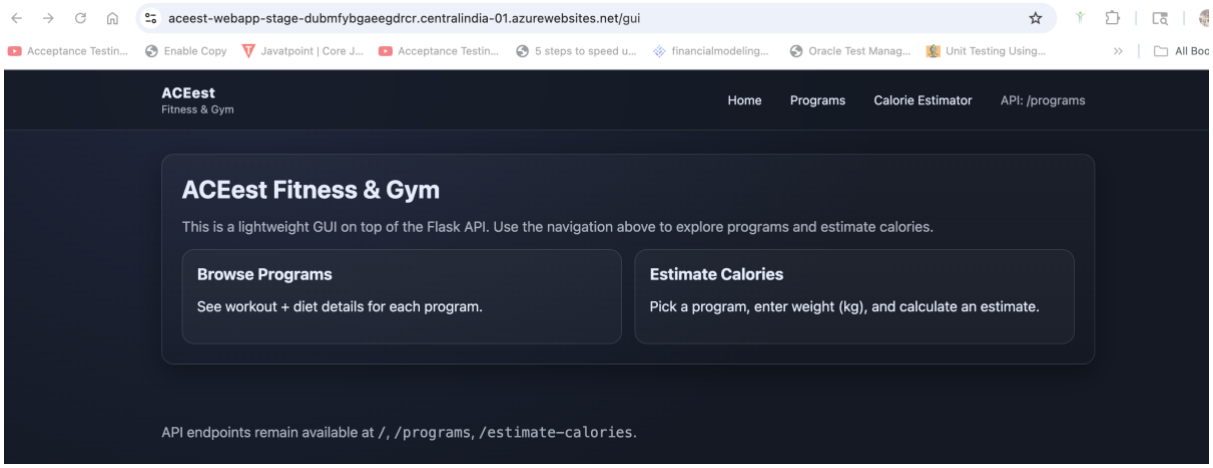
Returns the ACEest API health message.



Programs API: https://aceest-webapp-stage-dubmfybgaeegdrcr.centralindia-01.azurewebsites.net/programs



GUI home: `https://aceest-webapp-stage-dubmfybgaeegdrcr.centralindia-01.azurewebsites.net/gui`



12. CI/CD PIPELINE DIAGRAM (LOGICAL FLOW)

Logical flow of the pipeline:

1. **Developer**

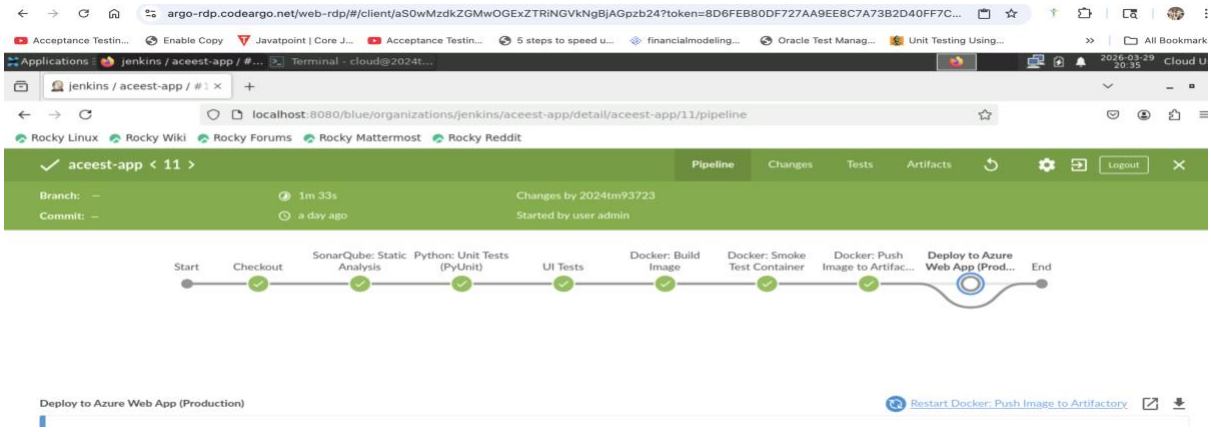
- Writes code, commits to feature branch, pushes to GitHub.

2. **GitHub (Repo + GitHub Actions)**

- Trigger on `push` / `pull_request`.
- Run Build & Lint → Docker Image Assembly → Pytest tests.

3. **Jenkins Pipeline**

- Poll SCM detects new commits / merges to `main` or feature branches.
- Stages:
  - Checkout
  - Static Code Analysis (Sonar CLI)
  - Pytest (and optional UI tests)
  - Docker Build
  - Docker Push to JFrog (stage/prod depending on branch)
  - Deploy to Azure Web App (for stage; similarly for prod if extended)



4. **JFrog Artifactory**

- Stores versioned Docker images:
  - `docker-local` for staging.
  - `docker-prod` for production.

5. **Azure Web App Deployment**

- Pulls the latest image from JFrog.
- Runs ACEest Fitness & Gym as a containerized web app.

13. CONCLUSION

The project demonstrates a real-world DevOps workflow integrating automation, testing, containerization, and deployment across multiple tools:

- **GitHub -** for version control, branching, PRs, and GitHub Actions CI.
- **GitHub Actions -** for repository-native CI (build, lint, Docker build, Pytest) on every push and PR.
- **Jenkins -** for extended CI/CD, static analysis, artifact management, and deployment.
- **Docker -** for consistent runtime environments.
- **JFrog Artifactory -** for artifact storage and image promotion.
- **Azure Web Apps -** for cloud hosting of the containerized application.